

TP PHP - Transmission de donnees entre pages

Introduction	3
Exercice 1 - Transmission via l'URL (GET)	4
1.1 Objectif	4
1.2 Fonctionnement de la methode GET	4
1.3 Code - employes.php	4
1.4 Code - employe.php	5
1.5 Explications	5
1.6 Captures d'ecran	5
Exercice 2 - Formulaire HTML et methode POST	6
2.1 Objectif	6
2.2 Differences entre GET et POST	6
2.3 Code - bonjour.php (version finale avec radio)	6
2.4 Code - reponse.php	7
2.5 Explications	7
2.6 Captures d'ecran	8
Exercice 3 - Liste deroulante et transmission POST	9
3.1 Objectif	9
3.2 Code - service.php	9
3.3 Code - service_post.php	10
3.4 Explications	10
3.5 Captures d'ecran	10
Exercice 4 - Formulaire complet en page unique	11
4.1 Objectif	11
4.2 Code - machineCafe.php	11
4.3 Explications	12
4.4 Captures d'ecran	13
Exercice 5 - Sécurisation contre les injections JavaScript (XSS)	14
5.1 Objectif	14
5.2 Exemple d'injection	14
5.3 Solution : htmlspecialchars()	14
5.4 Application	14
5.5 Bonne pratique	15
Conclusion	16

Introduction

Ce TP porte sur la transmission de données entre pages PHP. Il couvre plusieurs mécanismes fondamentaux du développement web côté serveur : le passage de paramètres via l'URL (méthode GET), la soumission de formulaires (méthode POST), et la sécurisation des entrées utilisateurs contre les injections JavaScript.

Les fichiers sont à placer dans le dossier `transmission-donnees/` accessible via XAMPP à l'adresse `http://localhost/transmission-donnees/`.

Recapitulatif des exercices

N	Fichier(s)	Objectif
Ex. 1	employees.php, employe.php	Passage de paramètres dans l'URL (méthode GET)
Bonus 1	employees-bis.php	Génération d'une liste par boucle avec paramètre GET
Ex. 2	bonjour.php, reponse.php	Formulaire HTML + POST + checkbox + boutons radio
Bonus 2	bonjour-bis.php	Saisie et affichage dans la même page
Ex. 3	service.php, service_post.php	Liste déroulante (<select>) et transmission POST
Ex. 4	machineCafe.php	Formulaire complet (texte, select, radio) en page unique
Ex. 5	Tous les fichiers	Sécurisation : protection contre les injections JS (XSS)

Exercice 1 - Transmission via l'URL (GET)

1.1 Objectif

Comprendre le passage de parametres dans l'URL avec la methode GET. La page employes.php affiche une liste de 3 employes, chacun avec un lien pointant vers employe.php?id=<identifiant>. La page employe.php recupere ce parametre et affiche l'identifiant reçu.

1.2 Fonctionnement de la methode GET

Lorsqu'un lien de la forme employe.php?id=2 est clique, le navigateur envoie une requete HTTP GET vers le serveur. PHP rend ce parametre accessible via le tableau superglobal \$_GET. La syntaxe est : \$_GET['id'].

Exemple d'URL generee : localhost/transmission-donnees/employe.php?id=1

1.3 Code - employes.php

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Liste des employés</title>
</head>
<body>
  <h1>Liste des employés</h1>
  <ul>
    <li>Employé 1 : <a href="employe.php?id=1">détails</a></li>
    <li>Employé 2 : <a href="employe.php?id=2">détails</a></li>
    <li>Employé 3 : <a href="employe.php?id=3">détails</a></li>
  </ul>
</body>
</html>
```

1.4 Code - employe.php

```
<!DOCTYPE html>

<html>

<head>

    <meta charset="UTF-8">

    <title>Détails sur un employé</title>

</head>

<body>

    <h1>Détails sur un employé</h1>

    <?php

        $id = $_GET['id'];

        echo "<p>Son identifiant est " . $id . "</p>";

    ?>

</body>

</html>
```

1.5 Explications

- employes.php : page statique HTML generant 3 liens avec le parametre id=1, id=2, id=3.
- employe.php : page PHP qui lit \$_GET['id'] et affiche sa valeur dans la page.
- Le tableau \$_GET est automatiquement rempli par PHP avec tous les parametres presents dans l'URL apres le signe ?.

1.6 Captures d'écran

Liste des employés

- Employé 1 : [détails](#)
- Employé 2 : [détails](#)
- Employé 3 : [détails](#)

Détails sur un employé

Son identifiant est 2

Exercice 2 - Formulaire HTML et methode POST

2.1 Objectif

Créer un formulaire de saisie avec la méthode POST, récupérer les valeurs côté serveur, et afficher une réponse personnalisée. Le formulaire est enrichi progressivement avec une case à cocher (checkbox) et des boutons radio.

2.2 Differences entre GET et POST

Critere	GET	POST
Donnees	Dans l'URL (visibles)	Dans le corps de la requete (invisibles)
Taille	Limitee (~2000 caracteres)	Illimitee
Superglobal PHP	\$_GET['cle']	\$_POST['cle']
Usage typique	Recherche, navigation	Formulaire, envoi de donnees sensibles

2.3 Code - bonjour.php (version finale avec radio)

```
<!DOCTYPE html>

<html>

<head>

  <meta charset="UTF-8">

  <title>Bonjour</title>

</head>

<body>

  <form method="post" action="reponse.php">

    <label>Entrez votre prénom :</label>

    <input type="text" name="prenom" required>

    <br><br>

    <label>Cochez pour un bonjour plus familier :</label>

    <input type="checkbox" name="familier" value="oui">

    <br><br>

    <input type="radio" name="civilite" value="Mademoiselle"> Mademoiselle<br>
```

```

<input type="radio" name="civilite" value="Madame"> Madame<br>
<input type="radio" name="civilite" value="Monsieur" checked> Monsieur<br>
<br>

<input type="submit" value="Dire bonjour">
</form>
</body>
</html>

```

2.4 Code - reponse.php

```

<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Réponse</title>
</head>
<body>
  <?php
    $prenom = htmlspecialchars($_POST['prenom']);
    $familier = isset($_POST['familier']);
    $civilite = htmlspecialchars($_POST['civilite']);

    $salutation = $familier ? 'Salut' : 'Bonjour';
    $formule = strtolower($civilite);

    echo "<p>$salutation $formule $prenom !</p>";
  ?>
</body>
</html>

```

2.5 Explications

- method="post" : les données du formulaire sont envoyées via HTTP POST.

- action="reponse.php" : la page de destination qui traite les donnees.
- required sur l'input : validation HTML5 obligeant la saisie du prenom.
- La checkbox n'est transmise dans \$_POST que si elle est cochee. On utilise isset() pour detecter sa presence.
- Les boutons radio partagent le meme attribut name="civilite" : un seul peut etre selectionne a la fois.
- L'operateur ternaire (\$familier ? 'Salut' : 'Bonjour') choisit le type de salutation selon la case cochee.

2.6 Captures d'ecran

Entrez votre prénom :

Cochez pour un bonjour plus familier : ☐

- ☐ Mademoiselle
☐ Madame
☒ Monsieur

Bonjour monsieur fahad !

Entrez votre prénom :

Cochez pour un bonjour plus familier : ☒

- ☐ Mademoiselle
☐ Madame
☒ Monsieur

Salut monsieur fahad !

Exercice 3 - Liste deroulante et transmission POST

3.1 Objectif

Créer un formulaire avec une liste deroulante (<select>) proposant des services avec leurs codes. Le service selectionne est envoye via POST et son code est affiche dans une page de reponse.

3.2 Code - service.php

```
<!DOCTYPE html>

<html>

<head>

    <meta charset="UTF-8">

    <title>Sélection du service</title>

</head>

<body>

    <h1>Choix d'un service</h1>

    <form method="post" action="service_post.php">

        <label>Service :</label>

        <select name="service">

            <option value="s01">Administration</option>

            <option value="s02">Commercial</option>

            <option value="s03">Emballage</option>

            <option value="s04">Fabrication</option>

        </select>

        <br><br>

        <input type="submit" value="Sélectionner">

    </form>

</body>

</html>
```

3.3 Code - service_post.php

```
<!DOCTYPE html>

<html>

<head>

    <meta charset="UTF-8">

    <title>Service sélectionné</title>

</head>

<body>

    <?php

        $code = htmlspecialchars($_POST['service']);

        echo "<p>Le code du service choisi est $code</p>";

    ?>

</body>

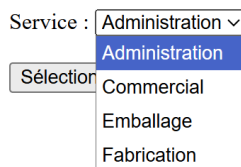
</html>
```

3.4 Explications

- La balise <select name="service"> cree la liste deroulante.
- Chaque option a un attribut value qui est la valeur reellement transmise (code du service).
- Le texte visible entre les balises <option> est l'intitule affiche a l'utilisateur.
- La valeur selectionnee est recuperee via \$_POST['service'] dans la page de traitement.

3.5 Captures d'ecran

Choix d'un service



Le code du service choisi est s01

Exercice 4 - Formulaire complet en page unique

4.1 Objectif

Réaliser un formulaire complet (machine à café) dans une seule page machineCafe.php. Le formulaire comporte : un champ texte Nom, un champ texte Prenom, une liste déroulante de boissons, et des boutons radio Sucre / Sans sucre. Le résultat est affiché dans la même page après validation.

4.2 Code - machineCafe.php

```
<!DOCTYPE html>

<html>

<head>

    <meta charset="UTF-8">

    <title>Machine à café</title>

</head>

<body>

    <h1>Votre boisson</h1>

    <form method="post" action="">

        <label>Nom :</label>

        <input type="text" name="nom" required>

        <br><br>

        <label>Prénom :</label>

        <input type="text" name="prenom" required>

        <br><br>

        <label>Choisissez votre boisson :</label>

        <select name="boisson">

            <option value="Café">Café</option>

            <option value="Thé">Thé</option>

            <option value="Chocolat">Chocolat</option>

            <option value="Café crème">Café crème</option>

        </select>

        <br><br>
```

```

        <input type="radio" name="sucre" value="Oui" checked> Sucre<br>
        <input type="radio" name="sucre" value="Non"> Sans sucre<br>
        <br>

        <input type="submit" name="valider" value="Valider">
        <input type="reset" value="Annuler">
    </form>

<?php
    if (isset($_POST['valider'])) {
        $nom      = htmlspecialchars($_POST['nom']);
        $prenom   = htmlspecialchars($_POST['prenom']);
        $boisson  = htmlspecialchars($_POST['boisson']);
        $sucre    = htmlspecialchars($_POST['sucre']);

        echo "<p>Bonjour $prenom $nom</p>";
        echo "<p>Votre choix : $boisson</p>";
        echo "<p>Sucre : $sucre</p>";
    }

?>
</body>
</html>

```

4.3 Explications

- `action=""` : le formulaire s'envoie sur lui-meme (page unique).
- `name="valider"` sur le bouton submit : permet de detecter le clic avec `isset($_POST['valider'])`.
- `<input type="reset">` : bouton Annuler qui remet le formulaire a zero sans recharger la page.
- Les boutons radio Sucre / Sans sucre partagent le meme `name="sucre"`.
- La condition `if (isset($_POST['valider']))` garantit que l'affichage n'apparait que si le formulaire a ete soumis.

4.4 Captures d'ecran

Votre boisson

Nom :

Prénom :

Choisissez votre boisson :

- ☒ Sucre
☐ Sans sucre

Bonjour fahad mohammed

Votre choix : Chocolat

Sucre : Oui

Votre boisson

Nom :

Prénom :

Choisissez votre boisson :

- ☒ Sucre
☐ Sans sucre

Exercice 5 - Sécurisation contre les injections JavaScript (XSS)

5.1 Objectif

Securiser tous les formulaires precedents contre les attaques de type XSS (Cross-Site Scripting). Une injection JavaScript consiste a saisir du code malveillant dans un champ de formulaire pour l'executer dans le navigateur d'un autre utilisateur.

5.2 Exemple d'injection

Sans protection, si un utilisateur saisit le texte suivant dans un champ prenom :

```
<script>alert('Injection XSS !')</script>
```

Ce code serait interprete par le navigateur et une boite d'alerte s'afficherait. Dans un scenario reel, le code injecte pourrait voler des cookies de session ou rediriger l'utilisateur vers un site malveillant.

5.3 Solution : htmlspecialchars()

La fonction PHP htmlspecialchars() convertit les caracteres speciaux HTML en entites HTML afin qu'ils soient affiches comme du texte brut et non interpretes comme du code.

Caractere	Brut (dangereux)	Avec htmlspecialchars()
<	< (ouvre une balise)	<
>	> (ferme une balise)	>
"	" (attribut HTML)	"
&	& (entite HTML)	&

5.4 Application

Chaque affichage de donnee provenant d'un utilisateur (GET ou POST) doit etre protege. Le principe est simple : remplacer toute occurrence de `echo $variable;` par `echo htmlspecialchars($variable);`.

Exemple - employe.php securise

```
<?php
    $id = $_GET['id'];
    echo "<p>Son identifiant est " . $id . "</p>";
?>
```

Exemple - reponse.php securise

```
<?php
    $prenom    = htmlspecialchars($_POST['prenom']);
    $familier  = isset($_POST['familier']);
    $civilite  = htmlspecialchars($_POST['civilite']);

    $salutation = $familier ? 'Salut' : 'Bonjour';
    $formule    = strtolower($civilite);

    echo "<p>$salutation $formule $prenom !</p>";
?>
```

Exemple - machineCafe.php securise

```
<?php
    if (isset($_POST['valider'])) {
        $nom      = htmlspecialchars($_POST['nom']);
        $prenom   = htmlspecialchars($_POST['prenom']);
        $boisson  = htmlspecialchars($_POST['boisson']);
        $sucre    = htmlspecialchars($_POST['sucre']);

        echo "<p>Bonjour $prenom $nom</p>";
        echo "<p>Votre choix : $boisson</p>";
        echo "<p>Sucre : $sucre</p>";
    }
?>
```

5.5 Bonne pratique

- Toujours appliquer htmlspecialchars() avant tout affichage de donnée non maîtrisée.
- La sécurisation doit être appliquée à la sortie (output), pas à l'entrée.
- Le paramètre ENT_QUOTES peut être ajouté pour échapper aussi les apostrophes : htmlspecialchars(\$var, ENT_QUOTES, 'UTF-8').

Bonjour monsieur <script>alert('XSS')</script> !

Conclusion

Ce TP a permis d'aborder les différents mécanismes de transmission de données entre pages web en PHP :

- La méthode GET pour transmettre des paramètres simples dans l'URL, facilement lisibles mais limités en taille et visibles.
- La méthode POST pour envoyer des données de formulaire de façon plus discrète, adaptée aux données sensibles ou volumineuses.
- L'utilisation des différents éléments de formulaire HTML : champs texte, liste déroulante, case à cocher, boutons radio.
- La gestion d'une page en auto-référence (action vide), permettant de réunir saisie et affichage dans un seul fichier.
- La sécurisation des entrées utilisateurs via `htmlspecialchars()` pour prévenir les attaques XSS.

Ces concepts sont fondamentaux dans le développement web côté serveur et constituent la base de toute interaction entre un utilisateur et une application PHP.